

Project report on

AURDINO POWERED LCD DINO GAME:AN ENDLESS RUNNER GAME EXPERIENCE

Submitted by

Abhishek Sahoo (2241019026)

Abhay Kumar Singh (2241019452)

B. Tech. (CSE(IOT) 5th Semester (Section-2241037)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Institute of Technical Education and Research

**SIKSHA 'O' ANUSANDHAN
DEEMED TO BE UNIVERSITY**

Bhubaneswar, Odisha, India.
(Oct, 2024)

Abstract

The **Arduino-Powered LCD Dino Game** is an exciting endless runner game designed with the Arduino Nano microcontroller and a basic LCD display. The project replicates the well-known dinosaur game, where a pixelated dinosaur must run and jump over obstacles to survive. Players use external buttons to control the character's movement, and the game progressively increases in difficulty as the dino encounters more challenging obstacles. This project exemplifies how basic electronics and programming can be combined to create a dynamic gaming experience with minimal hardware requirements.

At the heart of this game is the Arduino Nano, which handles input processing and game logic, while the LCD screen provides visual feedback. The game is programmed to respond in real time, ensuring smooth gameplay as the player attempts to achieve high scores by avoiding oncoming obstacles. The minimalist design makes it accessible to hobbyists and beginner programmers, yet the challenge of creating fluid and responsive gameplay on limited hardware introduces deeper learning opportunities in embedded systems and game development.

This project not only serves as a fun and interactive game but also as a learning tool for understanding how microcontrollers interact with displays and user inputs. Its simplicity allows for easy modifications and expansions, enabling users to experiment with new features like scoring systems, sound effects, or more complex obstacles. The Arduino-Powered LCD Dino Game is an excellent example of how creative applications of technology can produce engaging, real-world results while strengthening foundational electronics and coding skills.

Contents

Abstract.....	i
Contents.....	ii
Chapter 01: Introduction.....	1
1.1. Introduction.....	1
1.2. Background.....	1
1.3. Project Objectives	1
1.4. Scope	1
1.5. Project Management	1
1.6. Overview and Benefits.....	2
1.7. Organization of the Report.....	2
Chapter 02: Background Review & Survey	3
2.1. Related Works	3
Chapter 03: Theoretical Aspects.....	4
3.1. Internet of Things (IoT).....	4
3.2. Features of IoT	4
3.3. Advantages of IoT	5
3.4. Disadvantages of IoT	6
3.5. Application areas of IoT.....	7
3.6. IOT Technologies and Protocols	7
3.7. Project Layout.....	9
3.7.1. Brief Description.....	9
Chapter 04: Hardware Requirements	10
4.1. Arduino Uno.....	10
4.1.1. Features	10
4.1.2. Pin Configuration	10
4.2. ESP-01	10
4.3. Sensors	10
4.3.1. Pressure Sensor.....	10
4.3.2. Temperature Sensor	11
4.4.....	11
4.5. Block diagram of the proposed system	11
4.5.1. Working of the system	11

4.5.2. Circuit Diagram	11
4.5.3. Components Required	13
Chapter 05: Software Requirements.....	13
5.1. Arduino IDE (Embedded C / C++)	13
5.2. Logic and Flowchart	13
Chapter 06: Project development & Testing Aspects	14
6.1.....	Error! Bookmark not defined.
Chapter 07: Conclusion & Future Scope	16
7.1. Result.....	Error! Bookmark not defined.
7.2. Conclusion	16
7.3. Limitations	16
7.4. Further Enhancement and Future Scope	17
References	17
Appendix 01	17
A01.1. Code Listing	18
A01.2. Main Code	18
A01.3. Libraries	27
Appendix 02	27
A02.1. Project Proposal Form.....	27
A02.2. Project Management	28
A02.3. Bill of Material	28
Appendix 03	28
A03.1. Data Sheets	29

Chapter 01: Introduction

1.1. Introduction

The **Arduino-Powered LCD Dino Game** is an endless runner project that features a pixelated dinosaur navigating obstacles on an LCD screen. Controlled by external buttons, the game demonstrates the use of the Arduino Nano for real-time gameplay, offering a fun, interactive experience while introducing fundamental concepts of embedded systems and game development.

1.2 Background

- ❖ Inspired by the offline dinosaur game from web browsers, this project brings the endless runner concept to a hardware-based platform using Arduino.
- ❖ Utilizes an Arduino Nano, LCD screen, and external buttons to create a simple yet engaging game experience with real-time input handling and display output.
- ❖ Designed to teach foundational concepts in embedded systems, such as handling inputs, controlling LCD displays, and implementing game logic.
- ❖ Targeted at beginner programmers and electronics enthusiasts to provide hands-on experience in game development using minimal components.
- ❖ Encourages creative customization and further feature expansions.

1.3 Project Objectives

The Arduino-powered LCD Dino Game is an endless runner experience that challenges players to guide a dinosaur across obstacles using simple controls. Powered by an Arduino Nano, the game features responsive gameplay on an LCD screen, engaging players with increasing difficulty and speed, providing a fun, retro-style arcade experience.

1.4 Scope

The scope of the Arduino-powered LCD Dino Game project includes designing and programming the game on an Arduino Nano, creating dynamic obstacles, and implementing user input controls for jumping or dodging. The project will focus on LCD graphics, game mechanics, speed progression, and ensuring a seamless, interactive endless runner experience.

1.5 Project Management

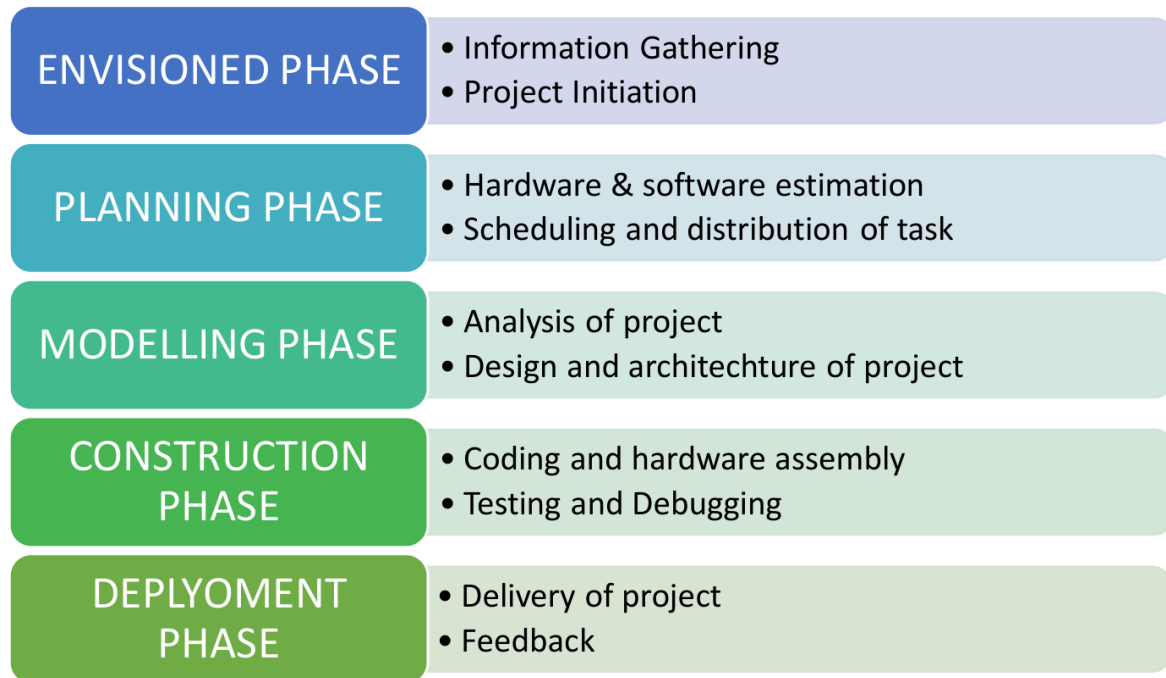


Figure 1. Model of phases in project management.

1.5 Overview and Benefits

The Arduino-powered LCD Dino Game offers a compact, retro-style endless runner experience, enhancing coding skills and hardware integration. It provides an engaging, portable gaming platform, fostering creativity in electronics while demonstrating the fundamentals of game design and programming.

1.6 Organization of the Report

The report is organised into the following chapters. Each chapter is unique on its own and is described with necessary theory to comprehend it.

Chapter 2 deals with background survey and review, Chapter 3 has the description of the theoretical aspects that has been acquired to commence the project work.

Chapter 02: Background Review & Survey

2.2 Related Works

- ❖ **Simple Arduino Games:** Projects that utilize Arduino boards to create basic games, such as "Arduino Pong" or "Arduino Snake," serving as foundational models for game design principles and user interaction.
- ❖ **Game Development Tutorials:** Online resources and tutorials that focus on programming games using Arduino, providing insights into graphics rendering, input handling, and game mechanics.
- ❖ **Retro Game Emulators:** Projects that emulate classic arcade games on microcontrollers, exploring the nostalgia of retro gaming and how to implement similar mechanics in a DIY format.
- ❖ **Obstacle Avoidance Projects:** Previous works that involve sensors and obstacle detection, which can be integrated into an endless runner game to enhance gameplay dynamics.
- ❖ **LCD Screen Applications:** Various projects showcasing the use of LCD displays with Arduino, demonstrating how to manage graphics and animations effectively for engaging user interfaces.

These related works provide inspiration, methodologies, and technical guidance for developing the endless runner game, highlighting the versatility of Arduino in gaming and interactive applications.

Chapter 03: Theoretical Aspects

3.2 Internet of Things (IoT)

The Internet of Things (IoT) refers to the network of interconnected devices and objects that can communicate and exchange data over the internet without human intervention. These devices, equipped with sensors and connectivity features, enable remote monitoring, control, and automation across various domains, enhancing efficiency and convenience.

3.3 Features of IoT

a) Intelligence

In the Arduino-powered LCD Dino Game, IoT features enable remote monitoring of player performance and adaptive gameplay. By collecting data on user interactions and scores, the game can intelligently adjust difficulty levels and provide updates, enhancing engagement and personalizing the experience while fostering a community through online leaderboards.

b) Connectivity

The Arduino-powered LCD Dino Game leverages IoT connectivity to allow seamless communication between the game and external devices. Players can connect via Bluetooth or Wi-Fi to upload scores, receive game updates, and participate in online challenges, fostering a community experience and enhancing engagement through real-time interactions and shared leaderboards.

c) Dynamic Nature

The IoT-enabled dynamic nature of the Arduino-powered LCD Dino Game allows real-time adaptation to player performance. The game can adjust difficulty levels, speed, and obstacle patterns based on data analytics, providing a personalized experience. Additionally, dynamic updates and new content can be delivered remotely, keeping the gameplay fresh and engaging.

d) Enormous Scale

The feature of enormous scale in the Arduino-powered LCD Dino Game allows widespread connectivity across multiple devices and players. With cloud integration and real-time data collection, the game can support a large number

of users, enabling global scoreboards, multiplayer challenges, and continuous updates, creating a scalable gaming experience.

e) Sensing

The IoT sensing feature in the Arduino-powered LCD Dino Game allows the game to detect player inputs, environmental changes, or even motion through sensors like accelerometers or touch sensors. This real-time sensing enhances gameplay by enabling intuitive controls, such as tapping or tilting, providing a more immersive and responsive gaming experience.

f) Heterogeneity

The IoT feature of heterogeneity in the Arduino-powered LCD Dino Game enables it to interact with various devices and platforms, such as smartphones, PCs, or external sensors. This allows the game to integrate different hardware and communication protocols, supporting cross-platform play, diverse input methods, and enhanced connectivity, enriching the overall gaming experience.

g) Security

The IoT security feature in the Arduino-powered LCD Dino Game ensures safe data transmission and protection of player information. By incorporating encryption and secure communication protocols, the game safeguards player scores, personal data, and device connections, preventing unauthorized access, hacking, or tampering, thereby maintaining a secure and trusted gaming environment.

3.3 Advantages of IoT

a) Communication

Communication in the Arduino-powered LCD Dino Game project enables efficient collaboration, faster problem-solving, and improved design iteration. It helps share knowledge across hardware and software tasks, ensures alignment on goals, and keeps the project on track, resulting in a smoother development process and a successful final product.

b) Automation and Control

Automation and control in the Arduino-powered LCD Dino Game project ensure precise game mechanics, such as automatic speed adjustments, obstacle generation, and real-time score tracking. It enhances gameplay experience by providing consistent responses to player inputs, reducing manual intervention, and ensuring smooth, reliable performance throughout the game.

c) Information

Information in the Arduino-powered LCD Dino Game project enables real-time feedback, like displaying the score, game status, and player actions on the screen. This enhances player engagement by providing critical game data, guiding decisions during gameplay, and improving the overall experience with clear, timely updates.

d) Monitoring

Monitoring in the Arduino-powered LCD Dino Game project allows real-time tracking of game performance, player inputs, and system health. It ensures smooth operation by detecting issues like input lag or display errors early, enabling prompt adjustments and maintaining optimal gameplay, enhancing both reliability and user experience.

e) Efficiency

Efficiency in the Arduino-powered LCD Dino Game project ensures optimal use of resources, such as processing power and memory, enabling smooth gameplay without lag. Efficient code and hardware usage extend battery life, enhance responsiveness, and streamline game performance, resulting in a seamless and enjoyable gaming experience for users.

3.4 Disadvantages of IoT

a) Compatibility

A disadvantage of compatibility in the Arduino-powered LCD Dino Game project is the potential limitation in integrating external components, like advanced sensors or newer display modules. The Arduino platform may not support certain hardware or software features, restricting expansion and limiting the game's functionality or graphical performance.

b) Complexity

The disadvantage of complexity in the Arduino-powered LCD Dino Game project is that intricate coding and hardware configurations can make

development challenging. Managing game logic, real-time input handling, and display rendering increases the risk of bugs and delays, making the project harder to debug, maintain, and scale effectively.

c) Privacy/Security

Privacy and security concerns in the Arduino-powered LCD Dino Game project are minimal but can arise if external modules, like Wi-Fi or Bluetooth, are integrated. In such cases, weak security measures may expose the system to unauthorized access, potentially compromising user data or game settings. Proper safeguards are essential.

d) Safety

The disadvantage of safety in the Arduino-powered LCD Dino Game project arises from handling electronic components. Improper wiring, overheating, or incorrect power supply connections could pose risks of short circuits or hardware damage. Ensuring safe assembly and use of the components is essential to prevent electrical hazards during development and gameplay.

3.5 Application areas of IoT

Application areas of IoT in the Arduino-powered LCD Dino Game project can include remote game monitoring, player statistics tracking, and cloud-based leaderboards. Integrating IoT can enhance user engagement through real-time updates, allow for multiplayer functionality, and enable game analytics, making the experience more interactive and personalized for players.

3.6 IOT Technologies and Protocols

a) Bluetooth

Bluetooth in IoT technologies facilitates wireless communication between the Arduino-powered LCD Dino Game and mobile devices. This allows players to connect smartphones for enhanced controls, updates, or multiplayer interactions. Using Bluetooth protocols ensures secure and efficient data transfer, enhancing the gaming experience through real-time interaction and customizable features.

b) Zigbee

Zigbee in IoT technologies provides a low-power, wireless communication protocol suitable for connecting multiple devices in the Arduino-powered LCD Dino Game project. It enables efficient data exchange among sensors and controllers, supporting features like multiplayer capabilities and remote

monitoring, while ensuring scalability and robustness in the gaming environment.

c) Z-Wave

Z-Wave in IoT technologies offers a reliable, low-power wireless communication protocol ideal for connecting devices in the Arduino-powered LCD Dino Game project. It allows seamless integration of smart devices for enhanced gameplay features, such as remote control or automation, ensuring stable performance and extended battery life in the gaming setup.

d) Wi-Fi

Wi-Fi in IoT technologies enables high-speed wireless communication for the Arduino-powered LCD Dino Game, allowing seamless connectivity to the internet. This facilitates online features like cloud-based leaderboards, real-time multiplayer interactions, and software updates, enhancing user engagement and providing a more dynamic gaming experience through instant data exchange.

e) Cellular

Cellular technology in IoT provides mobile connectivity for the Arduino-powered LCD Dino Game, enabling gameplay and data exchange over long distances without relying on Wi-Fi. This allows players to access online features, such as remote monitoring and updates, enhancing the game's functionality and ensuring consistent performance regardless of location.

f) NFC

NFC (Near Field Communication) in IoT technologies allows for short-range, secure interactions in the Arduino-powered LCD Dino Game. Players can easily connect their devices for quick data transfer, such as saving progress or exchanging scores. This enhances user experience by enabling seamless, immediate interactions without complex pairing processes.

g) LoRaWAN

LoRaWAN (Long Range Wide Area Network) in IoT technologies enables long-distance, low-power communication for the Arduino-powered LCD Dino Game. This protocol allows remote data collection, such as player statistics or game status, from various locations, enhancing gameplay through persistent connectivity while conserving battery life and minimizing infrastructure requirements.

3.7 Project Layout

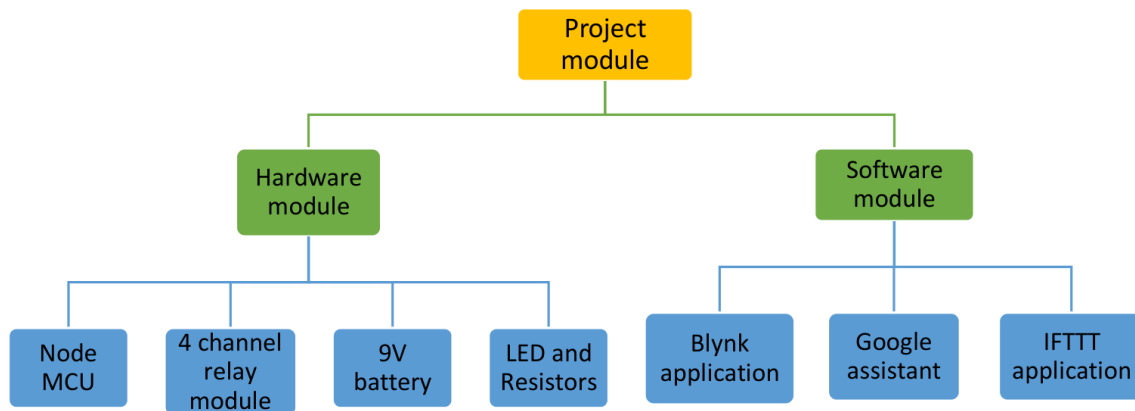


Figure 2. Layout of project module

3.7.1 Brief Description

The **Arduino-powered LCD Dino Game** is an engaging endless runner game designed to provide users with an interactive gaming experience using simple electronic components. Players control a dinosaur character displayed on an LCD screen, navigating through various obstacles while collecting points. The game utilizes the Arduino platform for processing inputs, managing game logic, and rendering graphics, making it an excellent project for those interested in embedded systems and programming. The setup typically includes buttons for player controls and an LCD display for visual feedback.

This project not only serves as a fun gaming experience but also offers valuable hands-on learning opportunities in electronics and coding. By working with sensors, displays, and the Arduino environment, users can enhance their understanding of how to create responsive, interactive applications. The game encourages creativity, allowing developers to modify features such as speed, difficulty, and graphics, fostering a deeper appreciation for game design and software development.

Chapter 04: Hardware Requirements

4.2 Arduino Uno

The Arduino Uno serves as the central microcontroller in the Arduino-powered LCD Dino Game, managing input from buttons and controlling the LCD display. Its versatility allows for real-time processing of game logic, enabling smooth gameplay and interactions. The Uno's simplicity makes it ideal for prototyping and educational projects.

4.2.1 Features

The Arduino Uno features multiple digital and analog input/output pins, allowing easy connection of buttons and sensors for player control in the LCD Dino Game. Its built-in microcontroller enables efficient processing of game logic, while compatibility with various libraries simplifies programming graphics and sound, enhancing the overall gaming experience.

4.2.2 Pin Configuration

In the Arduino-powered LCD Dino Game, the pin configuration of the Arduino Uno involves connecting buttons to digital input pins for player control and interfacing the LCD display with specific pins for data and control signals. This setup facilitates seamless communication and responsiveness in gameplay, ensuring smooth interactions.

4.3 ESP-01

The ESP-01 is a Wi-Fi module that can enhance the Arduino-powered LCD Dino Game by enabling wireless connectivity. It allows for features like online leaderboards and multiplayer capabilities, facilitating real-time data exchange. Its compact design and ease of integration make it an excellent addition for expanding game functionality.

4.4 Sensors

In the Arduino-powered LCD Dino Game, sensors can be used to enhance gameplay by detecting player actions, such as jumping or ducking. For instance, a distance sensor may trigger obstacles or modify game difficulty. Integrating sensors adds interactivity, creating a more immersive gaming experience for players.

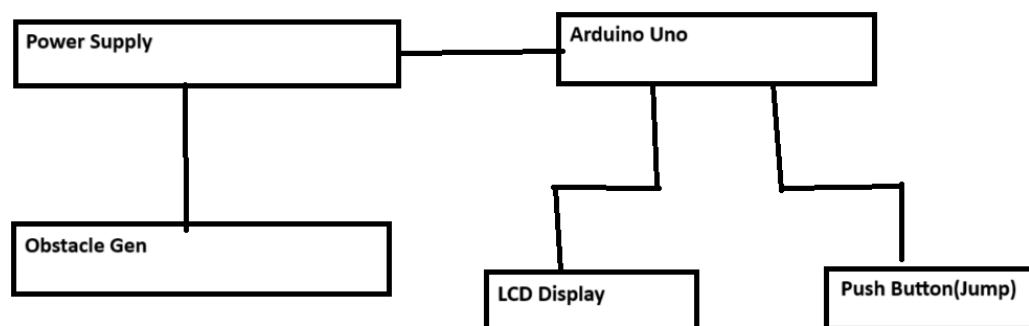
4.4.1 Pressure Sensor

In the Arduino-powered LCD Dino Game, a pressure sensor can detect player actions, such as pressing down harder to jump or perform special moves. This enhances interactivity and gameplay dynamics, allowing for more nuanced control. Integrating a pressure sensor can elevate the gaming experience by responding to varying input intensities.

4.4.2 Temperature Sensor

In the Arduino-powered LCD Dino Game, a temperature sensor can be integrated to monitor environmental conditions, potentially influencing gameplay elements. For example, higher temperatures could increase game speed or obstacle frequency. This adds a unique layer of interaction, making the game responsive to real-world conditions and enhancing player engagement.

4.5 Block diagram of the proposed system



4.5.2 Working of the system

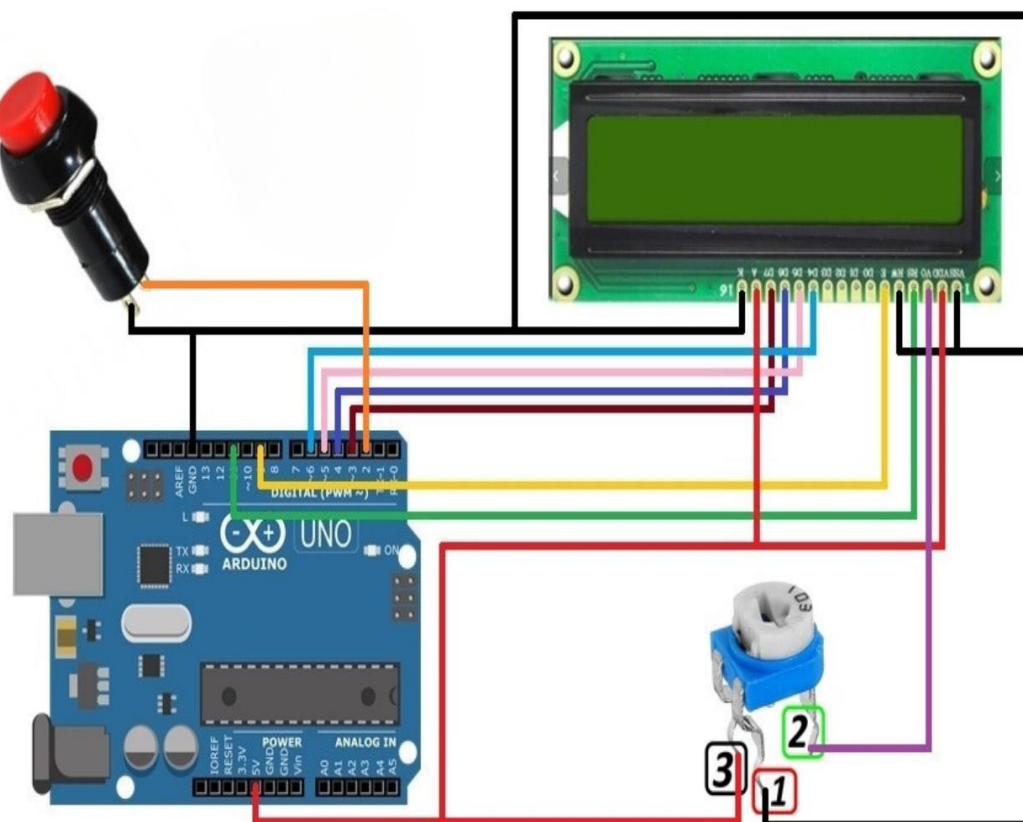
The **Arduino-powered LCD Dino Game** is designed as an endless runner where the player controls a Dino character using push buttons to either jump or duck. The game logic periodically generates obstacles that appear to move from right to left across the screen, while the Dino remains stationary. The player must press the button at the right time to make the Dino jump over or duck under the obstacles, with a collision leading to a game-over event. The game speed progressively increases, adding difficulty as more obstacles are avoided. The game also tracks the player's score based on the time survived or number of obstacles cleared, which can be displayed on the LCD screen.

An LCD display shows the character, obstacles, and score, with regular refreshes to create the illusion of motion. The buttons are connected to the Arduino's digital pins, triggering actions

like jumping or crouching when pressed. A piezo buzzer may be included to play sound effects when certain events like jumps or collisions occur. The game is powered by an Arduino board (such as Nano or Uno), and libraries like `LiquidCrystal.h` or `Adafruit_GFX.h` handle the display. The game's loop continually checks for button inputs, moves obstacles, checks for collisions, updates the score, and refreshes the screen to ensure smooth gameplay. Optional EEPROM memory can be used to store high scores.

The main game logic is handled in an infinite loop within the Arduino code. This loop continuously checks for user input, updates the position of the obstacles, checks for collisions, and manages the score. Each frame updates the LCD screen to simulate motion and keeps track of whether the player has successfully avoided or collided with an obstacle. Optional components, such as a piezo buzzer, can be used to produce sound effects for jumps, successful obstacle avoidance. Additionally, the game's high score feature can be implemented using the **EEPROM.h** library, which allows the Arduino to store and retrieve the highest score even when the device is powered off. This simple but engaging system creates an interactive and progressively challenging game experience using the Arduino platform.

4.5.3 Circuit Diagram



4.54 Components Required

Table 1. Component listing.

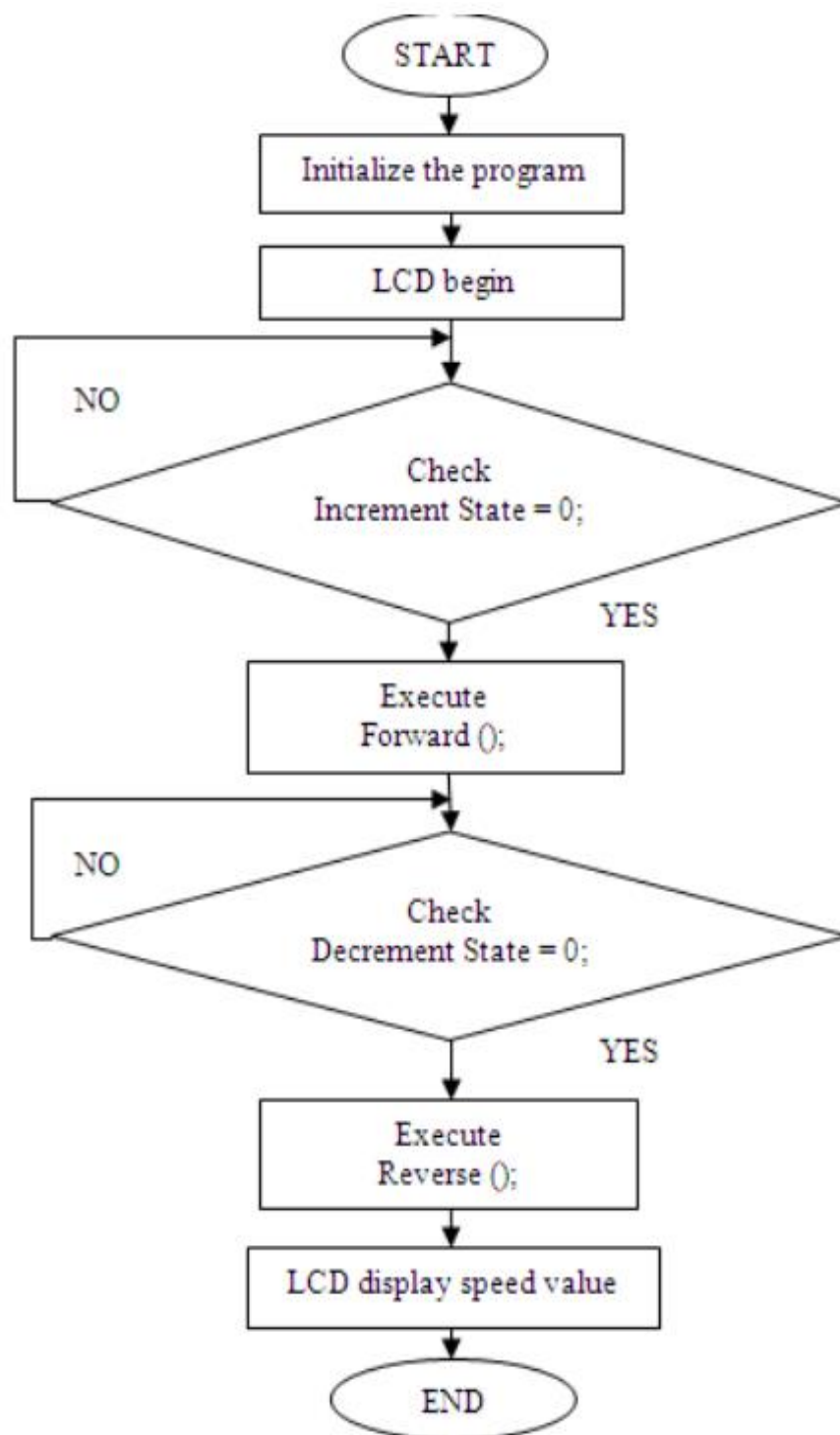
Sl. No.	Component and Specification	Quantity
1.	Arduino Uno	1
2.	Arduino Uno Cable	1
3.	Breadboard	1
4.	Jumper Wires (Male -Male)	20
5.	Jumper Wires (Male-Female)	20
6.	16×2 Character LCD	1
7.	Variable Register	1
8.	Push Button (Red Big)	1

Chapter 05: Software Requirements

5.4 Arduino IDE (Embedded C / C++)

The Arduino IDE is a user-friendly platform designed for writing, compiling, and uploading code to Arduino microcontrollers using Embedded C/C++. It supports a wide range of hardware libraries, enabling easy control of sensors, motors, and communication modules. The IDE simplifies the process of coding embedded systems by offering pre-configured settings, integrated libraries, and debugging tools. Its straightforward interface is ideal for both beginners and professionals, making it popular for projects in robotics, automation, IoT, and other embedded system designs.

5.3 Logic and Flow Chart



Chapter 06: Project development & Testing Aspects

The development of the Arduino-powered LCD Dino game begins with defining the game concept and selecting the necessary hardware. The key components include an Arduino Nano or Uno, an LCD screen (either 16x2 or graphical), and buttons for controlling player actions. These buttons are connected to the Arduino's digital pins, allowing the player to make the dino character jump and avoid obstacles. Optionally, a piezo buzzer is included to add sound effects. The game is powered via USB or battery, with the LCD wired for display output and the Arduino controlling all interactions. Ensuring solid electrical connections and power management is crucial for a stable system.

On the software side, the game logic is written in Embedded C/C++ using the Arduino IDE. Libraries such as Liquid Crystal (for basic LCDs) or U8glib (for graphical LCDs) manage the display, while custom code handles game functions like obstacle generation, player movement, and collision detection. The game operates in a continuous loop where the player must jump over randomly generated obstacles. As the player survives, the game increases in difficulty by speeding up obstacle movement. A scoring system tracks the player's progress, with scores increasing the longer the player avoids obstacles.

Testing and optimization are essential to ensure a smooth experience. This includes testing button responsiveness, refining the LCD refresh rate to minimize flicker, and balancing the game's difficulty. Additional testing checks for performance over extended periods to ensure no issues arise with prolonged gameplay. Optional features such as sound effects using a buzzer can be integrated to improve user engagement. The final step involves polishing the game's performance, ensuring responsive controls, and maintaining smooth gameplay as difficulty increases over time.

Chapter 07: Conclusion & Future Scope

7.1 Result

The Arduino-powered LCD Dino game resulted in a compact, engaging endless runner experience where a dinosaur character avoids obstacles on a scrolling screen. Built using the Arduino Nano and an LCD display, the game showcases simple yet effective pixel graphics, creating a retro, arcade-style feel. The gameplay is smooth, with responsive controls that allow players to jump over or dodge incoming objects, keeping the game endlessly challenging.

This project highlights the capabilities of the Arduino Nano in handling real-time input and display updates. Despite hardware limitations, the game provides a fun, minimalistic gaming experience, demonstrating how embedded systems can be used creatively to produce interactive entertainment. The project is a great example of blending hardware and software to create a classic, enjoyable

7.4 Conclusion

The Arduino-powered LCD Dino Game is an engaging endless runner that showcases the potential of microcontroller-based gaming. By leveraging the Arduino Nano and a simple LCD screen, it delivers a minimalist yet immersive experience, emphasizing the balance between creativity and technical implementation in game design.

7.5 Limitations

The Arduino-powered LCD Dino Game has limitations, including low graphical resolution due to the basic LCD display and restricted processing power of the Arduino Nano. These constraints limit game complexity, animation smoothness, and sound integration, making it challenging to expand beyond simple gameplay mechanics and features.

7.6 Further Enhancement and Future Scope

The Arduino-powered LCD Dino Game can be enhanced by integrating sound effects, improved graphics, and smoother animations. Using a more advanced microcontroller or an upgraded display could enhance the visual quality, while adding sound modules would make the game more engaging. Introducing sensors like accelerometers for motion-based controls or adding multiple difficulty levels can improve interactivity and increase replay value.

In the future, the project could evolve into a multi-level platformer by expanding the game's environment and features. Incorporating wireless connectivity, such as Bluetooth or Wi-Fi, could enable multiplayer functionality or online scoreboards. Additionally, designing a more modular system with easily interchangeable components would open up new possibilities for educational and hobbyist communities to customize and extend the game, promoting learning in embedded systems and game development.

In terms of future scope, the project could evolve into a multi-level platformer or even be expanded into a multiplayer game by adding wireless connectivity (Bluetooth or Wi-Fi). This would allow players to compete with others, introducing an online leaderboard system to track global scores. Additionally, the game could be adapted for educational purposes, demonstrating microcontroller programming, basic electronics, and game design in classrooms. The modular nature of the project opens opportunities for hobbyists and educators to customize and expand the game further, fostering innovation and learning in embedded systems.

References

<https://www.youtube.com/shorts/ijXk2A4cci8>

<https://www.youtube.com/watch?v=LMWp1eWxj08>

<https://www.youtube.com/watch?v=tdUTZlMm6pM>

Appendix 01

A01.1. Code Listing

Arduino-powered LCD Dino Game: The game uses a 16x2 LCD, an Arduino Nano, and a button for the Dino to jump. The LCD is wired in 4-bit mode with RS to Pin 7, EN to Pin 6, and data pins (D4-D7) connected to Pins 5-2. A custom character is created for the Dino and obstacles (e.g., a cactus), displayed as the game progresses. The Dino starts on the bottom row and the player presses the button (connected to Pin 8) to jump. Obstacles move from right to left, and the Dino must jump to avoid collisions. The game checks for button input to initiate the jump, keeping the Dino in the air for a set duration. The player earns points for successfully dodging obstacles. If a collision is detected (Dino and obstacle in the same position), the game ends and the score is displayed. The main game loop handles obstacle movement, Dino jumping logic, scorekeeping, and game over conditions.

A01.2. Main Code

```
#include <LiquidCrystal_I2C.h>
```

```
#include <Wire .h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
#define PIN_BUTTON 2
```

```
#define PIN_AUTOPLAY 1
```

```
#define PIN_READWRITE 10
```

```
#define PIN_CONTRAST 7
```

```
#define SPRITE_RUN1 1
```

```
#define SPRITE_RUN2 2
```

```
#define SPRITE_JUMP 3
```

```
#define SPRITE_JUMP_UPPER ''
```

```
#define SPRITE_JUMP_LOWER 4
```

```
#define SPRITE_TERRAIN_EMPTY ''
```

```
#define SPRITE_TERRAIN_SOLID 5
```

```
#define SPRITE_TERRAIN_SOLID_RIGHT 6
```

```
#define SPRITE_TERRAIN_SOLID_LEFT 7
```

```
#define HERO_HORIZONTAL_POSITION 1
```

```
#define TERRAIN_WIDTH 16
```

```
#define TERRAIN_EMPTY 0
```

```
#define TERRAIN_LOWER_BLOCK 1
```

```
#define TERRAIN_UPPER_BLOCK 2
```

```
#define HERO_POSITION_OFF 0
```

```
#define HERO_POSITION_RUN_LOWER_1 1
```

```
#define HERO_POSITION_RUN_LOWER_2 2
```

```
#define HERO_POSITION_JUMP_1 3
```

```
#define HERO_POSITION_JUMP_2 4
```

```
#define HERO_POSITION_JUMP_3 5
```

```
#define HERO_POSITION_JUMP_4 6
```

```
#define HERO_POSITION_JUMP_5 7
```

```
#define HERO_POSITION_JUMP_6 8
```

```
#define HERO_POSITION_JUMP_7 9
```

```
#define HERO_POSITION_JUMP_8 10
```

```
#define HERO_POSITION_RUN_UPPER_1 11
```

```
#define HERO_POSITION_RUN_UPPER_2 12
```

```
static char terrain Upper [TERRAIN_WIDTH + 1];
```

```
static char terrain Lower [TERRAIN_WIDTH + 1];
```

```
static bool button Pushed = false;
```

```
void initialize Graphics () {
```

```
    static byte graphics [] =
```

```
{
```

```
    B11110, B10111, B11110, B11100, B11111, B11100, B10010, B11011,
```

```

B11110, B10111, B11110, B11100, B11111, B11100, B01010, B10011,

B11110, B10111, B11110, B11100, B11111, B11110, B10001, B00000,

B11111, B11100, B10010, B11011, B00000, B00000, B00000, B00000,

B00100, B11111, B00100, B00101, B11111, B00100, B00100, B00100,

B00000, B00011, B00000, B00000, B00011, B00000, B00000, B00000,

B00000, B11000, B00000, B01000, B11000, B00000, B00000, B00000,

B00000, B00000, B00000, B00000, B11110, B10111, B11110, B11100,
};

for (int i = 0; int i < 7; ++int i) {
    lcd. create Char( int i + 1, &graphics[i * 8]);
}

for (int i = 0; i < TERRAIN_WIDTH; ++i) {
    terrain Upper[i] = SPRITE_TERRAIN_EMPTY;
    terrain Lower[i] = SPRITE_TERRAIN_EMPTY;
}
}

void advance Terrain(char* terrain, byte new Terrain) {
    for (int i = 0; i < TERRAIN_WIDTH; ++i) {
        char current = terrain[i];
        char next = (i == TERRAIN_WIDTH - 1) ? new Terrain : terrain[i + 1];
        switch (current) {

```



```

    case SPRITE_TERRAIN_EMPTY:

        terrain[i] = (next == SPRITE_TERRAIN_SOLID) ? SPRITE_TERRAIN_SOLID_RIGHT :
        SPRITE_TERRAIN_EMPTY;

        break;

    case SPRITE_TERRAIN_SOLID:

        terrain[i] = (next == SPRITE_TERRAIN_EMPTY) ? SPRITE_TERRAIN_SOLID_LEFT :
        SPRITE_TERRAIN_SOLID;

        break;

    case SPRITE_TERRAIN_SOLID_RIGHT:

        terrain[i] = SPRITE_TERRAIN_SOLID;

        break;

    case SPRITE_TERRAIN_SOLID_LEFT:

        terrain[i] = SPRITE_TERRAIN_EMPTY;

        break;

    }

}

}

```

```

bool draw Hero(byte position, char* terrain Upper, char* terrain Lower, unsigned int score) {

    bool collide = false;

    char upper Save = terrain Upper[HERO_HORIZONTAL_POSITION];

    char lower Save = terrain Lower[HERO_HORIZONTAL_POSITION];

    byte upper, lower;

    switch (position) {

        case HERO_POSITION_OFF:

            upper = lower = SPRITE_TERRAIN_EMPTY;

            break;

        case HERO_POSITION_RUN_LOWER_1:

            upper = SPRITE_TERRAIN_EMPTY;

            lower = SPRITE_RUN1;

```

```
    break;
case HERO_POSITION_RUN_LOWER_2:
    upper = SPRITE_TERRAIN_EMPTY;
    lower = SPRITE_RUN2;
    break;
case HERO_POSITION_JUMP_1:
case HERO_POSITION_JUMP_8:
    upper = SPRITE_TERRAIN_EMPTY;
    lower = SPRITE_JUMP;
    break;
case HERO_POSITION_JUMP_2:
case HERO_POSITION_JUMP_7:
    upper = SPRITE_JUMP_UPPER;
    lower = SPRITE_JUMP_LOWER;
    break;
case HERO_POSITION_JUMP_3:
case HERO_POSITION_JUMP_4:
case HERO_POSITION_JUMP_5:
case HERO_POSITION_JUMP_6:
    upper = SPRITE_JUMP;
    lower = SPRITE_TERRAIN_EMPTY;
    break;
case HERO_POSITION_RUN_UPPER_1:
    upper = SPRITE_RUN1;
    lower = SPRITE_TERRAIN_EMPTY;
    break;
case HERO_POSITION_RUN_UPPER_2:
    upper = SPRITE_RUN2;
    lower = SPRITE_TERRAIN_EMPTY;
    break;
}
```

```
if (upper != SPRITE_TERRAIN_EMPTY) {  
    terrain Upper[HERO_HORIZONTAL_POSITION] = upper;  
    collide = (upper Save != SPRITE_TERRAIN_EMPTY);  
}
```

```
if (lower != SPRITE_TERRAIN_EMPTY) {  
    terrain Lower[HERO_HORIZONTAL_POSITION] = lower;  
    collide |= (lower Save != SPRITE_TERRAIN_EMPTY);  
}
```

```
byte digits = (score > 9999) ? 5 : (score > 999) ? 4 : (score > 99) ? 3 : (score > 9) ? 2 : 1;
```

```
terrain Upper[TERRAIN_WIDTH] = '\0';
```

```
terrain Lower[TERRAIN_WIDTH] = '\0';
```

```
char temp = terrain Upper[16 - digits];
```

```
terrain Upper[16 - digits] = '\0';
```

```
lcd.setCursor(0, 0);
```

```
lcd.print(terrain Upper);
```

```
terrain Upper[16 - digits] = temp;
```

```
lcd.setCursor(0, 1);
```

```
lcd.print(terrain Lower);
```

```
lcd.setCursor(16 - digits, 0);
```

```
lcd.print(score);
```

```
terrain Upper[HERO_HORIZONTAL_POSITION] = upper Save;
```

```
terrain Lower[HERO_HORIZONTAL_POSITION] = lower Save;
```

```
return collide;
```

```
}
```

```

void button Push () {
    button Pushed = true;
}

void setup() {
    pin Mode (PIN_READWRITE, OUTPUT);
    digital Write (PIN_READWRITE, LOW);
    pin Mode (PIN_CONTRAST, OUTPUT);
    digital Write (PIN_CONTRAST, LOW);
    pin Mode (PIN_BUTTON, INPUT_PULLUP);
    pin Mode (PIN_AUTOPLAY, OUTPUT);
    digital Write (PIN_AUTOPLAY, HIGH);

    lcd. Init ();
    lcd. backlight ();

    attach Interrupt (digital Pin To Interrupt (PIN_BUTTON), button Push, FALLING);

    initialize Graphics ();

    lcd . begin (16, 2);
}

void loop() {
    static byte hero Pos = HERO_POSITION_RUN_LOWER_1;
    static byte new Terrain Type = TERRAIN_EMPTY;
    static byte new Terrain Duration = 1;
    static bool playing = false;
    static bool blink = false;
    static unsigned int distance = 0;

```

```

if (!playing) {
    draw Hero((blink) ? HERO_POSITION_OFF : hero Pos, terrain Upper, terrain Lower,
distance >> 3);
    if (blink) {
        lcd.setCursor(0, 0);
        lcd.print("Press Start");
    }
    delay(250);
    blink = !blink;
    if (button Pushed) {
        initialize Graphics();
        hero Pos = HERO_POSITION_RUN_LOWER_1;
        playing = true;
        button Pushed = false;
        distance = 0;
    }
    return;
}

```

```

Advance Terrain (terrain Lower, new Terrain Type == TERRAIN_LOWER_BLOCK ?
SPRITE_TERRAIN_SOLID : SPRITE_TERRAIN_EMPTY);

```

```

Advance Terrain (terrain Upper, new Terrain Type == TERRAIN_UPPER_BLOCK ?
SPRITE_TERRAIN_SOLID : SPRITE_TERRAIN_EMPTY);

```

```

if (--new Terrain Duration == 0) {
    if (new Terrain Type == TERRAIN_EMPTY) {
        new Terrain Type = (random (3) == 0) ? TERRAIN_UPPER_BLOCK :
TERRAIN_LOWER_BLOCK;
        new Terrain Duration = 2 + random (10);
    } else {
        New Terrain Type = TERRAIN_EMPTY;
        new Terrain Duration = 10 + random (10);
    }
}

```

```
}  
}
```

```
if (button Pushed) {  
    if (hero Pos <= HERO_POSITION_RUN_LOWER_2) hero Pos =  
HERO_POSITION_JUMP_1;  
    button Pushed = false;  
}
```

```
if (draw Hero (hero Pos, terrain Upper, terrain Lower, distance >> 3)) {  
    playing = false;  
} else {  
    if (hero Pos == HERO_POSITION_RUN_LOWER_2 || hero Pos ==  
HERO_POSITION_JUMP_8) {  
        hero Pos = HERO_POSITION_RUN_LOWER_1;  
    } else if ((hero Pos >= HERO_POSITION_JUMP_3 && hero Pos <=  
HERO_POSITION_JUMP_5) && terrain Lower [HERO_HORIZONTAL_POSITION] !=  
SPRITE_TERRAIN_EMPTY) {  
        Hero Pos = HERO_POSITION_RUN_UPPER_1;  
    } else if (hero Pos >= HERO_POSITION_RUN_UPPER_1 && terrain Lower  
[HERO_HORIZONTAL_POSITION] == SPRITE_TERRAIN_EMPTY) {  
        Hero Pos = HERO_POSITION_JUMP_5;  
    } else if (hero Pos == HERO_POSITION_RUN_UPPER_2) {  
        Hero Pos = HERO_POSITION_RUN_UPPER_1;  
    } else {  
        ++hero Pos;  
    }  
    ++distance;
```

```
    Digital Write (PIN_AUTOPLAY, terrain Lower [HERO_HORIZONTAL_POSITION + 2] ==  
SPRITE_TERRAIN_EMPTY ? HIGH : LOW);  
}
```

```
delay (100);  
}
```

A01.3. Libraries

- ❖ **Liquid Crystal. h** – Used for controlling LCD screens connected to the Arduino. It handles displaying characters and graphics on the LCD.
- ❖ **Adafruit GFX. h** – If you're using a graphical LCD or OLED, this library provides graphics primitives like drawing lines, shapes, and text.
- ❖ **Adafruit_PCD8544.h** or **U8g2lib.h** – These libraries are used to drive certain types of graphical LCDs, such as the Nokia 5110 display (PCD8544) or other monochrome displays.
- ❖ **Bounce2.h** – For managing button presses (debouncing) more easily. In a game like Dino, you would need buttons to control the character, and this library can handle the debouncing of those inputs.
- ❖ **Tone. h** – If your game includes sound effects or alerts, this library can be used to generate sound with a piezo buzzer or speaker.
- ❖ **SPI. h** – If your game uses an LCD or display that communicates over SPI (Serial Peripheral Interface), this library would handle that communication.
- ❖ **EEPROM. h** – For storing high scores or other game data between power cycles, the EEPROM library allows writing and reading from the EEPROM memory on the Arduino.

Appendix 02

A02.1. Project Proposal Form

The project proposal form was prepared and duly signed from our Faculty-in-Charge Dr. Biswa Ranjan Swain. The same is attached at the last of this report.

A02.2 Project Management

#	Component	Individual Contributions in %		Total
		Abhishek	Abhay	
1.	Planning	50%	50%	100%
2.	Background Research and Analysis	50%	50%	100%
3.	Hardware design	30%	70%	100%
4.	Software design	80%	20%	100%
5.	Testing	50%	50%	100%
6.	Final Assembling	50%	50%	100%
7.	Project report writing	85%	15%	100%
8.	Presentation	50%	50%	100%
9.	Logistics	50%	50%	100%

A02.3. Bill of Material

Table 1. Component listing.

Sl. No.	Component	Specification	Unit Cost	Quantity	Total
1.	Arduino Uno		330.00	1	330.00
2.	Arduino Uno Cable		30.00	1	30.00
3.	Breadboard		70.00	1	70.00
4.	Jumper Wire	Male-Male	2.00	20	40.00
5.	Jumper Wire	Male-Female	2.00	20	40.00
6.	16×2 Character LCD		130.00	1	130.00
7.	Variable Register		10.00	1	10.00
8.	Push Button		15.00	1	15.00
9.	Gum & other accessories		500.00	1	500.00
	Grand Total				1245.00

Appendix 03

A03.1. Data Sheets

1. Arduino UNO & Arduino UNO cable

Data Sheet

In the Arduino-powered LCD Dino game, the Arduino UNO serves as the microcontroller that runs the game logic, handles button inputs, and controls the LCD display. The Arduino UNO cable is used to power the Arduino and upload the game code from a computer.

Specification

Arduino UNO Specifications:

- ❖ Microcontroller: The board is equipped with the ATmega328P microcontroller, providing the core processing power for running the game.
- ❖ Clock Speed: Operates at a clock speed of 16 MHz, ensuring timely execution of game logic.
- ❖ Digital Input/Output Pins: Contains 14 digital I/O pins, allowing for various sensor and control device connections.
- ❖ Analog Input Pins: Offers 6 Analog input pins to read Analog signals from sensors or other devices.
- ❖ Flash Memory: Has 32 KB of flash memory, with 0.5 KB reserved for the bootloader, enabling code storage and execution.
- ❖ Static Random-Access Memory (SRAM): Provides 2 KB of SRAM for temporary data storage during program execution.

Arduino UNO Cable Specifications:

- ❖ Type: The cable is a USB Type-B cable, commonly used for connecting devices to computers.
- ❖ Function: It serves dual purposes by powering the Arduino UNO and facilitating the upload of code from a computer to the board.

2. 16×2 Character LCD

Data Sheet

The **16×2 Character LCD** is used in the Arduino-powered LCD Dino game to display game graphics, including the player's character and terrain, as well as real-time information such as the player's score and game status. Its ability to show two rows of 16 characters each allows

for a clear and engaging presentation of game elements, enhancing the user experience during gameplay.

Specifications

The 16×2 Character LCD features a display with 16 columns and 2 rows, allowing it to show up to 32 characters simultaneously, making it ideal for presenting game status and scores clearly. It typically operates at a voltage of 5V, uses an HD44780 controller for easy interfacing with microcontrollers like the Arduino, and supports a wide range of characters and symbols, which can be customized for graphics in the game.

3. Breadboard

Data Sheet

The breadboard is utilized in the Arduino-powered LCD Dino game to facilitate the easy assembly and testing of the circuit components without the need for soldering, allowing for quick modifications and troubleshooting during development. It provides a flexible platform to connect the Arduino, LCD, and other electronic components, ensuring a reliable and organized setup for the game's hardware.

Specifications

The breadboard typically features a grid of interconnected metal clips under a plastic surface, enabling users to insert and connect various electronic components like resistors, capacitors, and ICs without soldering, facilitating rapid prototyping. Standard breadboards come in various sizes, with a common configuration of 830 tie points (63 rows and 5 columns) for versatile circuit designs, making it suitable for projects like the Arduino-powered LCD Dino game.

4. Jumper Wires

Data Sheet

Jumper wires are used in the Arduino-powered LCD Dino Game project to create connections between the Arduino UNO and various components such as the LCD, buttons, and power supply, facilitating communication and power distribution. They provide a flexible and easy way to prototype circuits on the breadboard, allowing for

quick adjustments and modifications during the game's development and testing phases.

Specifications

Jumper wires typically come in a variety of lengths and are available in male-to-male, female-to-female, or male-to-female configurations, making them versatile for different connections in the Arduino-powered LCD Dino Game project. These wires are usually insulated with a durable plastic coating, ensuring safe and reliable electrical connections between the Arduino UNO, breadboard, and other components, such as the LCD and buttons.

5. Push Button

Data Sheet

In the Arduino-powered LCD Dino Game project, a push button is used to control the game's actions, allowing the player to make the dinosaur jump by registering a button press. This input mechanism enhances interactivity and user engagement, enabling players to respond to obstacles in the game effectively.

Specifications

The push button used in the Arduino-powered LCD Dino Game typically features a momentary contact design, meaning it only completes the circuit while pressed, allowing for a responsive gaming experience. It usually operates at a low voltage (typically 5V for Arduino projects) and is compact in size, making it easy to integrate into breadboards or custom enclosures within the game setup.

6. Variable Resistors

Data Sheet

In the Arduino-powered LCD Dino Game project, a push button is used to control the game's actions, allowing the player to make the dinosaur jump by registering a button press. This input mechanism enhances interactivity and user engagement, enabling players to respond to obstacles in the game effectively.

Specifications

The push button used in the Arduino-powered LCD Dino Game typically features a momentary contact design, meaning it only completes the circuit while pressed, allowing for a responsive gaming experience. It usually operates at a low voltage (typically 5V for Arduino projects) and is compact in size, making it easy to integrate into breadboards or custom enclosures within the game setup.